

Intro to Data Science - HW 5

Copyright Jeffrey Stanton, Jeffrey Saltz, and Jasmina Tacheva

IST 687

Kent Roller

Due Date: 2/16/24

Submission Date: 2/16/24

```
# Enter your name here: Kent Roller
```

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

This module: Data visualization is important because many people can make sense of data more easily when it is presented in graphic form. As a data scientist, you will have to present complex data to decision makers in a form that makes the data interpretable for them. From your experience with Excel and other tools, you know that there are a variety of **common data visualizations** (e.g., pie charts). How many of them can you name?

Some of the types of visualizations that can be produced using R are Histograms, Scatterplots, Barplots, or line plots. Additionally for each of these types grouping can be applied to further enhance the visual.

The most powerful tool for data visualization in R is called **ggplot**. Written by computer/data scientist **Hadley Wickham**, this “**graphics grammar**” tool builds visualizations in layers. This method provides immense flexibility, but takes a bit of practice to master.

Step 1: Make a copy of the data

- A. Read the **who** dataset from this URL: <https://intro-datascience.s3.us-east-2.amazonaws.com/who.csv> (<https://intro-datascience.s3.us-east-2.amazonaws.com/who.csv>) into a new dataframe called **tb**.

Your new dataframe, `tb`, contains a so-called **multivariate time series**: a sequence of measurements on 23 Tuberculosis-related (TB) variables captured repeatedly over time (1980-2013). Familiarize yourself with the nature of the 23 variables by consulting the dataset's codebook which can be found here: https://intro-datascience.s3.us-east-2.amazonaws.com/TB_data_dictionary_2021-02-06.csv (https://intro-datascience.s3.us-east-2.amazonaws.com/TB_data_dictionary_2021-02-06.csv).

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.3      ✓ readr      2.1.4
## ✓ forcats    1.0.0      ✓ stringr    1.5.0
## ✓ ggplot2    3.4.4      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.0
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
url<-("https://intro-datascience.s3.us-east-2.amazonaws.com/who.csv")
```

```
tb<- read.csv(url(url))
```

B. How often were these measurements taken (in other words, at what frequency were the variables measured)? Put your answer in a comment.

```
# The measurements were taken once a year, or annually. One would imagine this represents the sum of the given years measurements for each age group and sex.
```

Step 2: Clean-up the NAs and create a subset

A. Let's clean up the `iso2` attribute in `tb`

Hint: use `is.na()` – well use `! is.na()`

```
tb<-tb[!is.na(tb$iso2),]
```

B. Create a subset of **tb** containing **only the records for Canada ("CA" in the iso2 variable)**. Save it in a new dataframe called **tbCan**. Make sure this new df has **29 observations and 23 variables**.

```
tb.can<-tb %>% filter(iso2 == "CA")
```

C. A simple method for dealing with small amounts of **missing data** in a numeric variable is to **substitute the mean of the variable in place of each missing datum**.

This expression locates (and reports to the console) all the missing data elements in the variable measuring the **number of positive pulmonary smear tests for male children 0-4 years old** (there are 26 data points missing)

```
tb.can$new_sp_m04[is.na(tb.can$new_sp_m04)]
```

```
## [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
## [26] NA
```

D. Write a comment describing how that statement works.

```
# The statement works by first accessing the specified variable using tb.can$new_sp_m04, then it calls the is.na function to sift through the variable and counting the NAs found inside, and we again specify the variable accessed for the count with the part in the ().
```

E. Write 4 more statements to check if there is missing data for the number of positive pulmonary smear tests for: **male and female** children 0-14 years old (**new_sp_m014** and **new_sp_f014**), and **male and female citizens 65 years of age and older**, respectively. What does empty output suggest about the number of missing observations?

```
tb.can$new_sp_m014[is.na(tb.can$new_sp_m014)]
```

```
## integer(0)
```

```
tb.can$new_sp_f014[is.na(tb.can$new_sp_f014)]
```

```
## integer(0)
```

```
tb.can$new_sp_m65[is.na(tb.can$new_sp_m65)]
```

```
## integer(0)
```

```
tb.can$new_sp_f65[is.na(tb.can$new_sp_f65)]
```

```
## integer(0)
```

There is an R package called **imputeTS** specifically designed to repair missing values in time series data. We will use this instead of mean substitution.

The **na_interpolation()** function in this package takes advantage of a unique characteristic of time series data: **neighboring points in time can be used to “guess” about a missing value in between.**

F. Install the **imputeTS** package (if needed) and use **na_interpolation()** on the variable from part C. Don't forget that you need to save the results back to the **tbCan** dataframe. Also update any attribute discussed in part E (if needed).

```
#install.packages("imputeTS")  
library(imputeTS)
```

```
## Warning: package 'imputeTS' was built under R version 4.3.2
```

```
## Registered S3 method overwritten by 'quantmod':  
##   method      from  
## as.zoo.data.frame zoo
```

```
tb.can$new_sp_m04<-na_interpolation(tb.can$new_sp_m04)
```

G. Rerun the code from C and E above to check that all missing data have been fixed.

```
tb.can$new_sp_m04[is.na(tb.can$new_sp_m04)]
```

```
## numeric(0)
```

```
tb.can$new_sp_m014[is.na(tb.can$new_sp_m014)]
```

```
## integer(0)
```

```
tb.can$new_sp_f014[is.na(tb.can$new_sp_f014)]
```

```
## integer(0)
```

```
tb.can$new_sp_m65[is.na(tb.can$new_sp_m65)]
```

```
## integer(0)
```

```
tb.can$new_sp_f65[is.na(tb.can$new_sp_f65)]
```

```
## integer(0)
```

Step 3: Use ggplot to explore the distribution of each variable

Don't forget to install and library the ggplot2 package. Then:

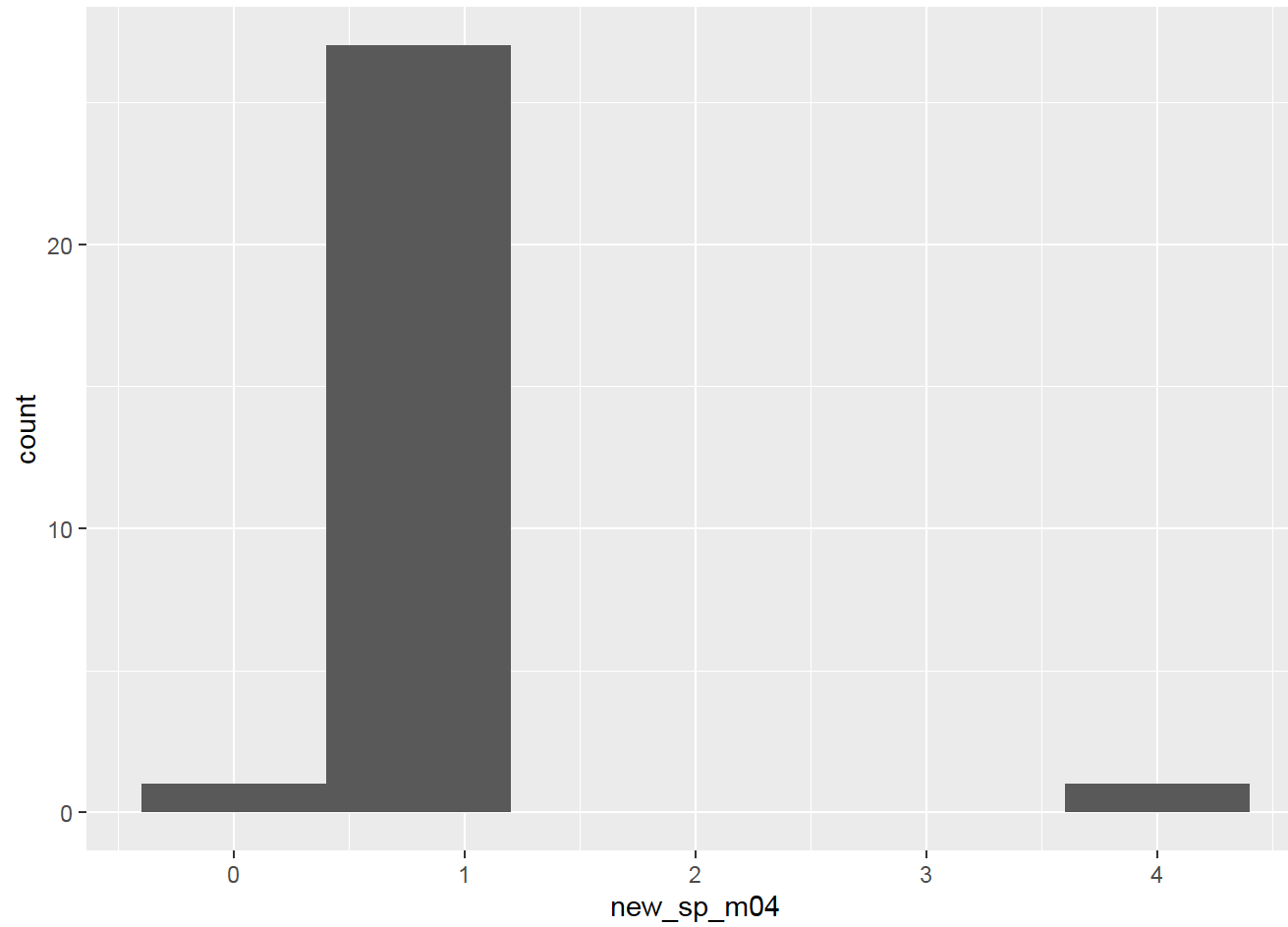
H. Create a histogram for **new_sp_m014**. Be sure to add a title and briefly describe what the histogram means in a comment.

```
library(ggplot2)
```

```
hist.m014<- ggplot(tb.can, aes(x=new_sp_m04)) + geom_histogram(bins=6, position="dodge")
```

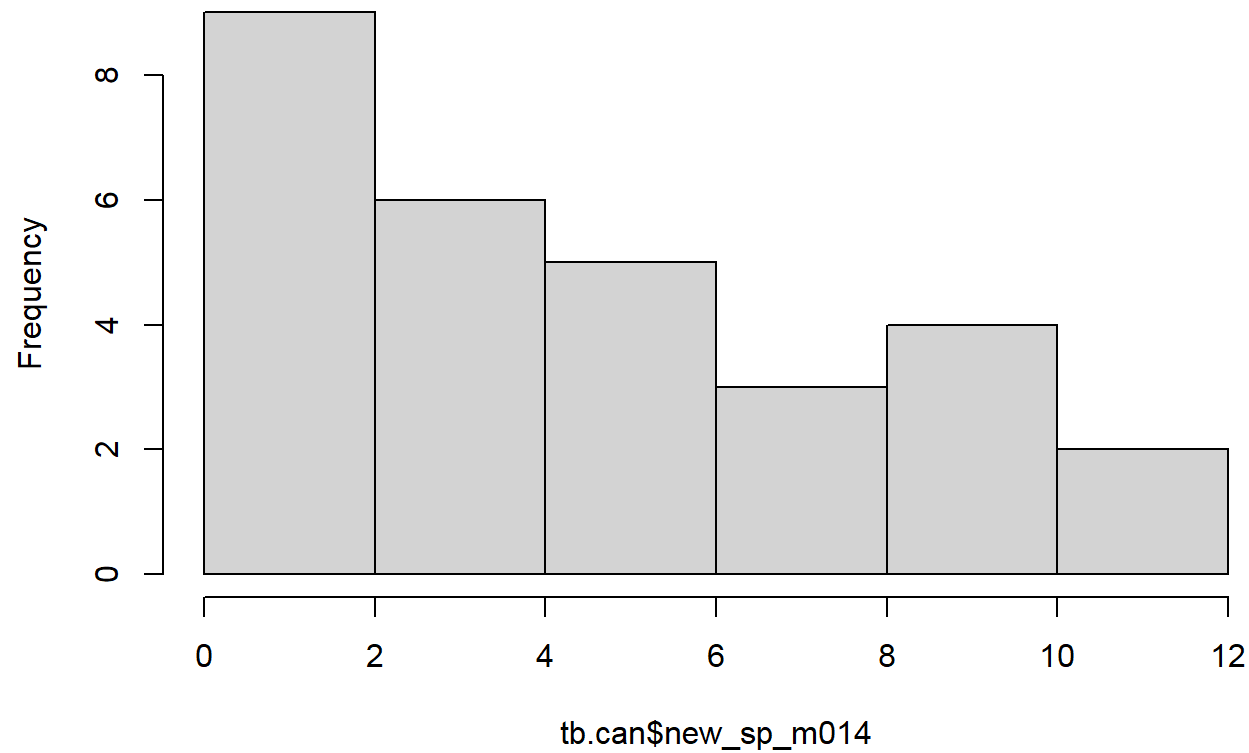
```
# I have tried several different things but cannot get the output from ggplot to match the output using hist(). Although I will say I am not entirely sure they should look the same but the fact that this was covered in the videos makes me feel that this is incorrect.
```

```
hist.m014
```

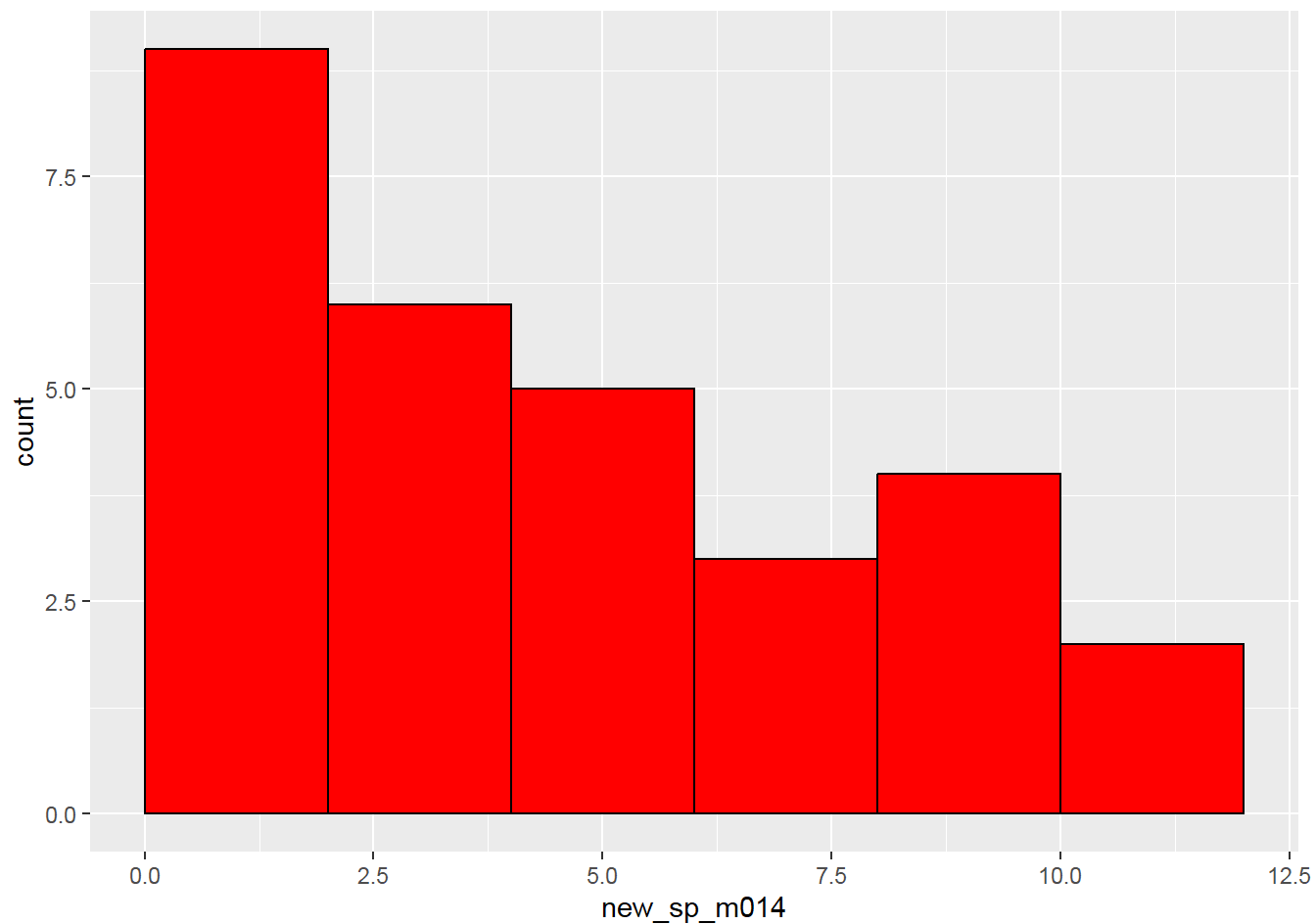


```
hist(tb.can$new_sp_m014)
```

Histogram of tb.can\$new_sp_m014



```
# I will try one more thing  
h.bins<-hist(tb.can$new_sp_m014, plot = FALSE)  
hist.m014.2<-ggplot(tb.can, aes(x=new_sp_m014)) + geom_histogram(breaks=h.bins$breaks, fill="red", color="black")  
hist.m014.2
```



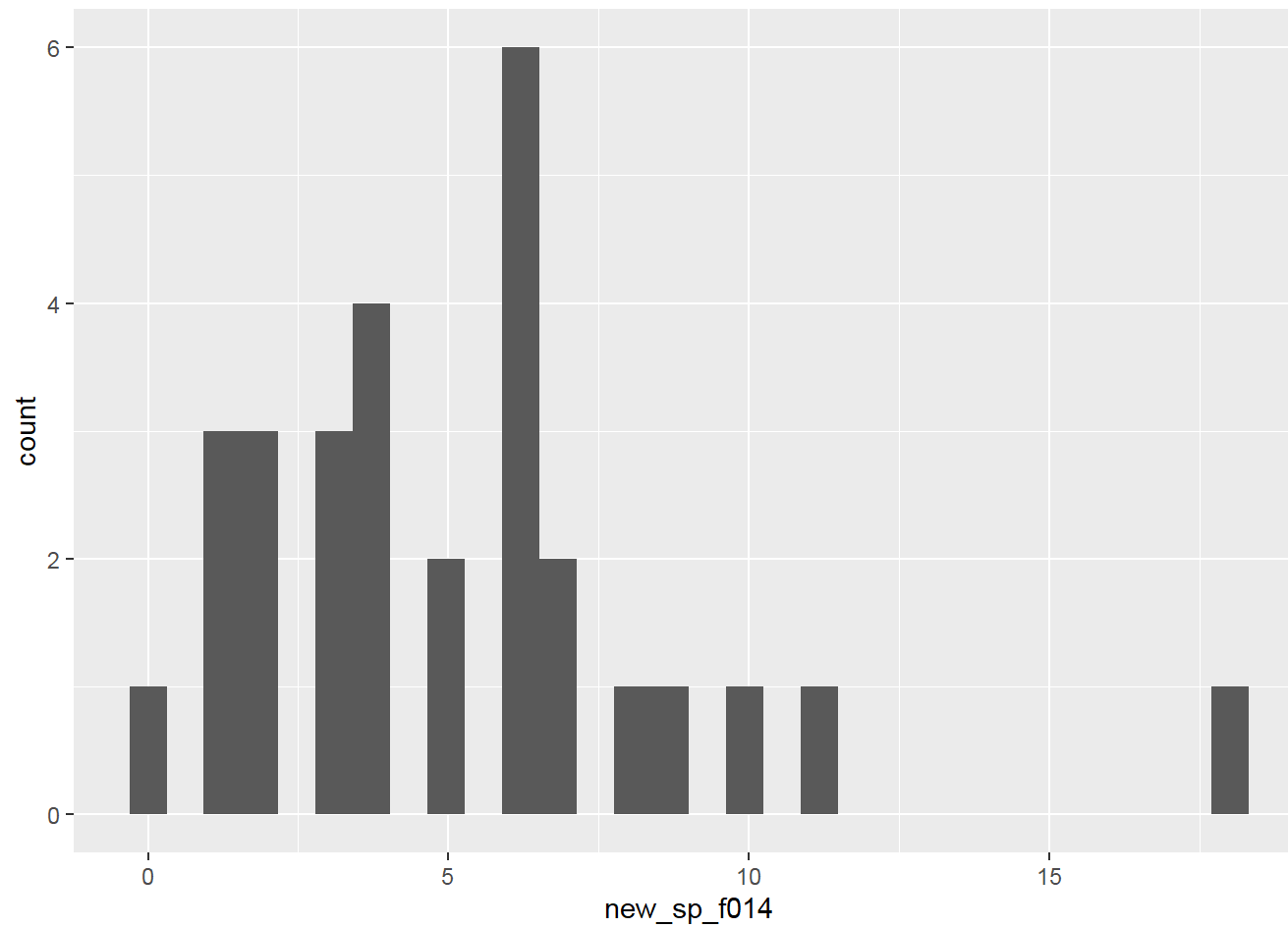
HA, not an elegant solution but this code uses the bin mapping from the hist() function and then uses that as the configuration for the output in the ggplot output.

I. Create histograms (using ggplot) of each of the other three variables from E with ggplot().

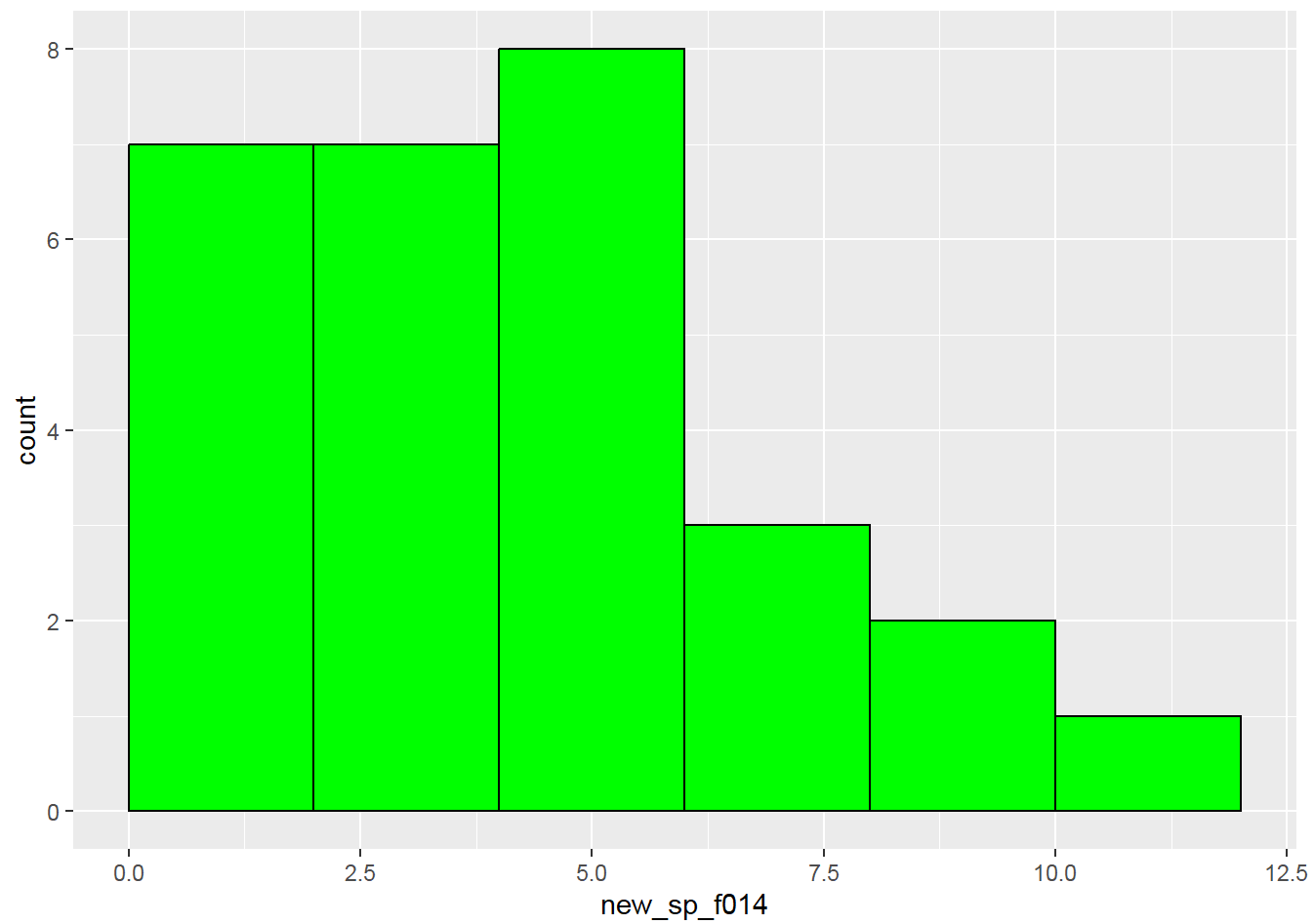
Which parameter do you need to adjust to make the other histograms look right? The bin spacing needs to be adjusted to make the histograms look similar to the format we are used to seeing (bins either together or stacked)

```
hist.f014<-ggplot(tb.can, aes(x=new_sp_f014)) + geom_histogram()
hist.f014
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

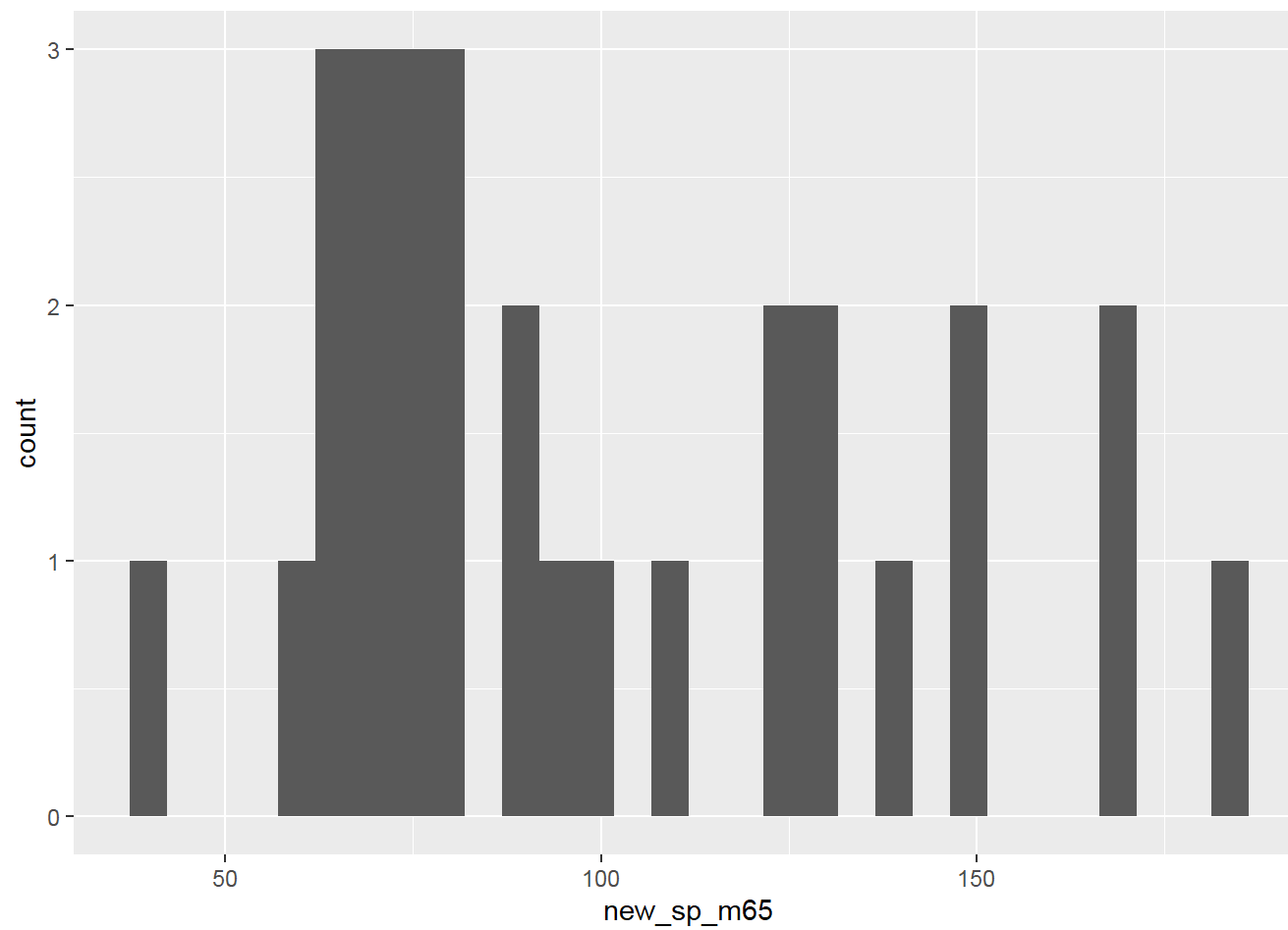


```
# Same problem as with the previous histogram, will try the same method to resolve.  
h.bins2<-hist(tb.can$new_sp_m014, plot = FALSE)  
hist.m014.2<-ggplot(tb.can, aes(x=new_sp_f014)) + geom_histogram(breaks=h.bins2$breaks, fill="green", color="black")  
hist.m014.2
```

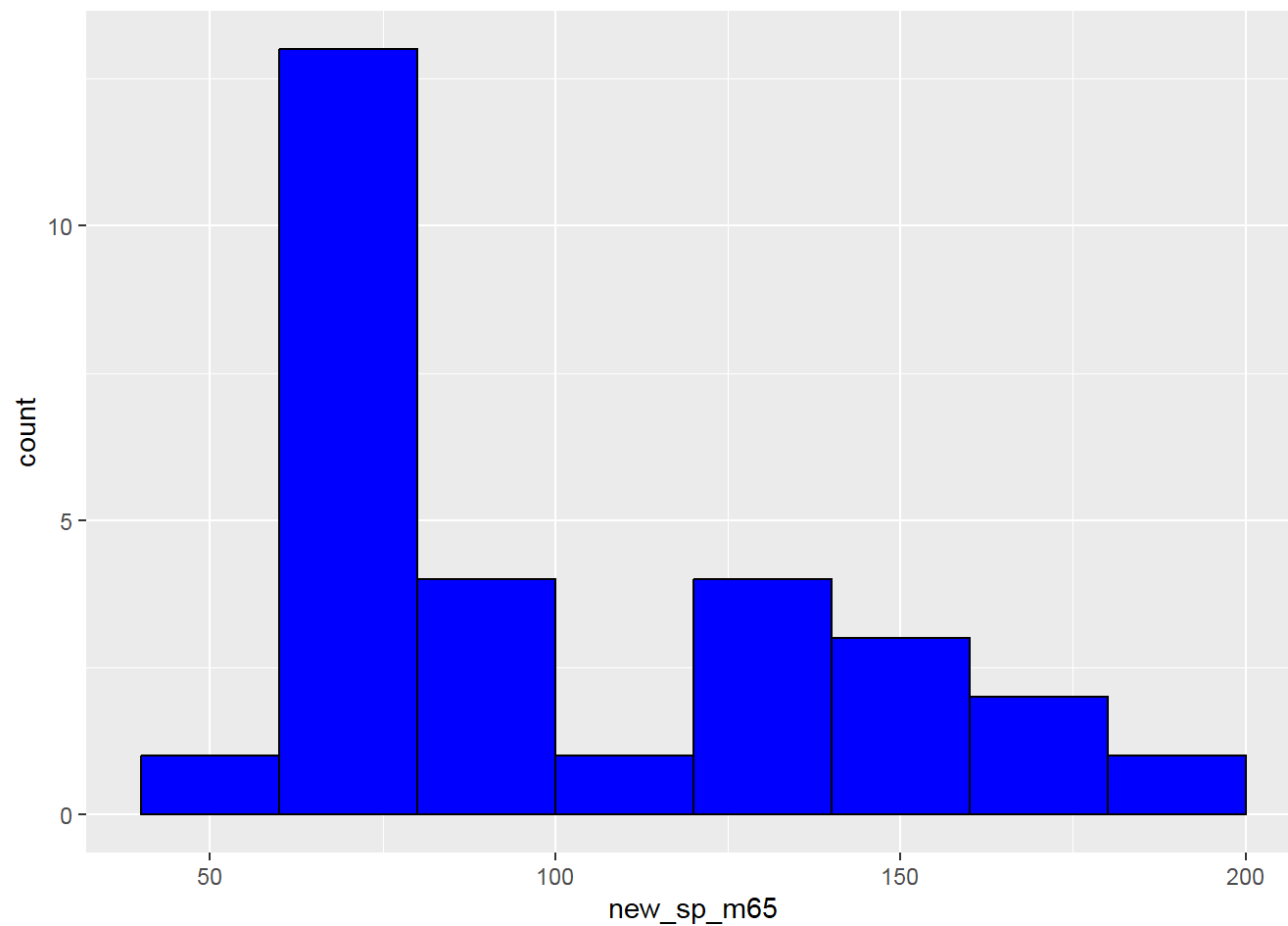


```
hist.m65<-ggplot(tb.can, aes(x=new_sp_m65)) + geom_histogram()  
hist.m65
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

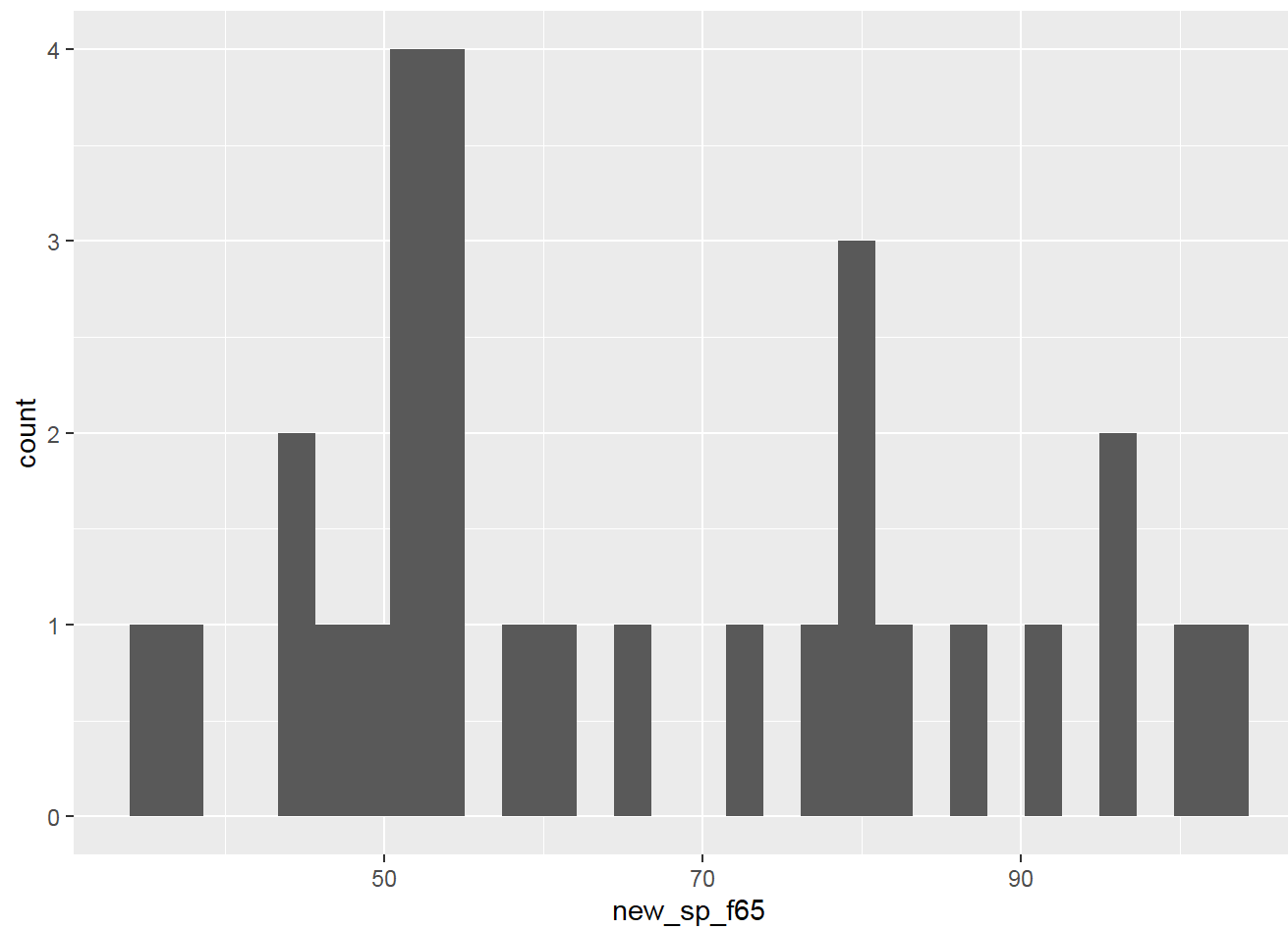


```
h.bins3<-hist(tb.can$new_sp_m65, plot= FALSE)
hist.m65.2<-ggplot(tb.can, aes(x=new_sp_m65)) + geom_histogram(breaks=h.bins3$breaks, fill="blue", color="black")
hist.m65.2
```

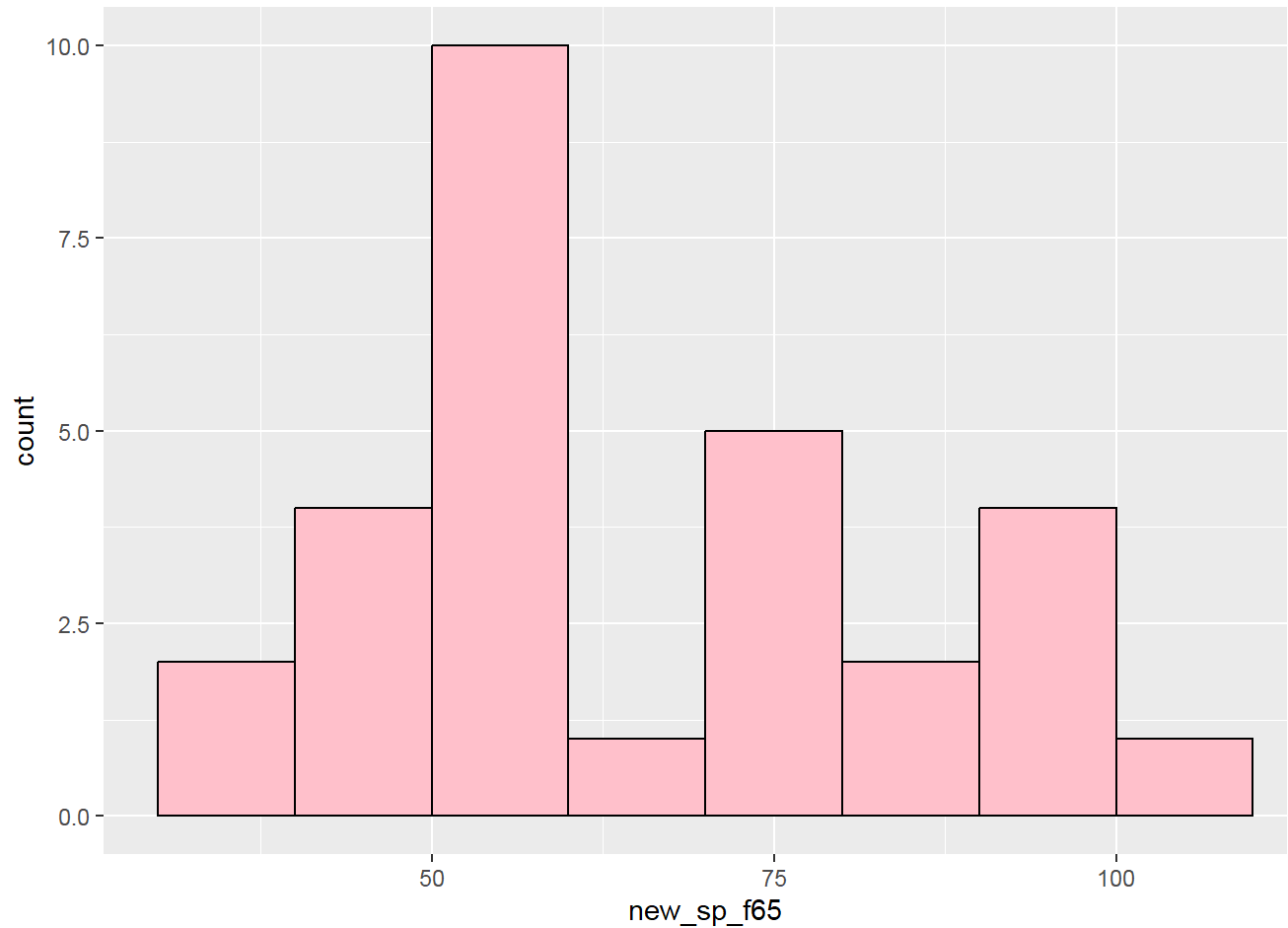


```
hist.f65<- ggplot(tb.can, aes(x=new_sp_f65)) + geom_histogram()  
hist.f65
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
h.bins4<-hist(tb.can$new_sp_f65, plot=FALSE)
hist.f65.2<-ggplot(tb.can, aes(x=new_sp_f65)) + geom_histogram(breaks=h.bins4$breaks, fill="pink",color="black" )
hist.f65.2
```



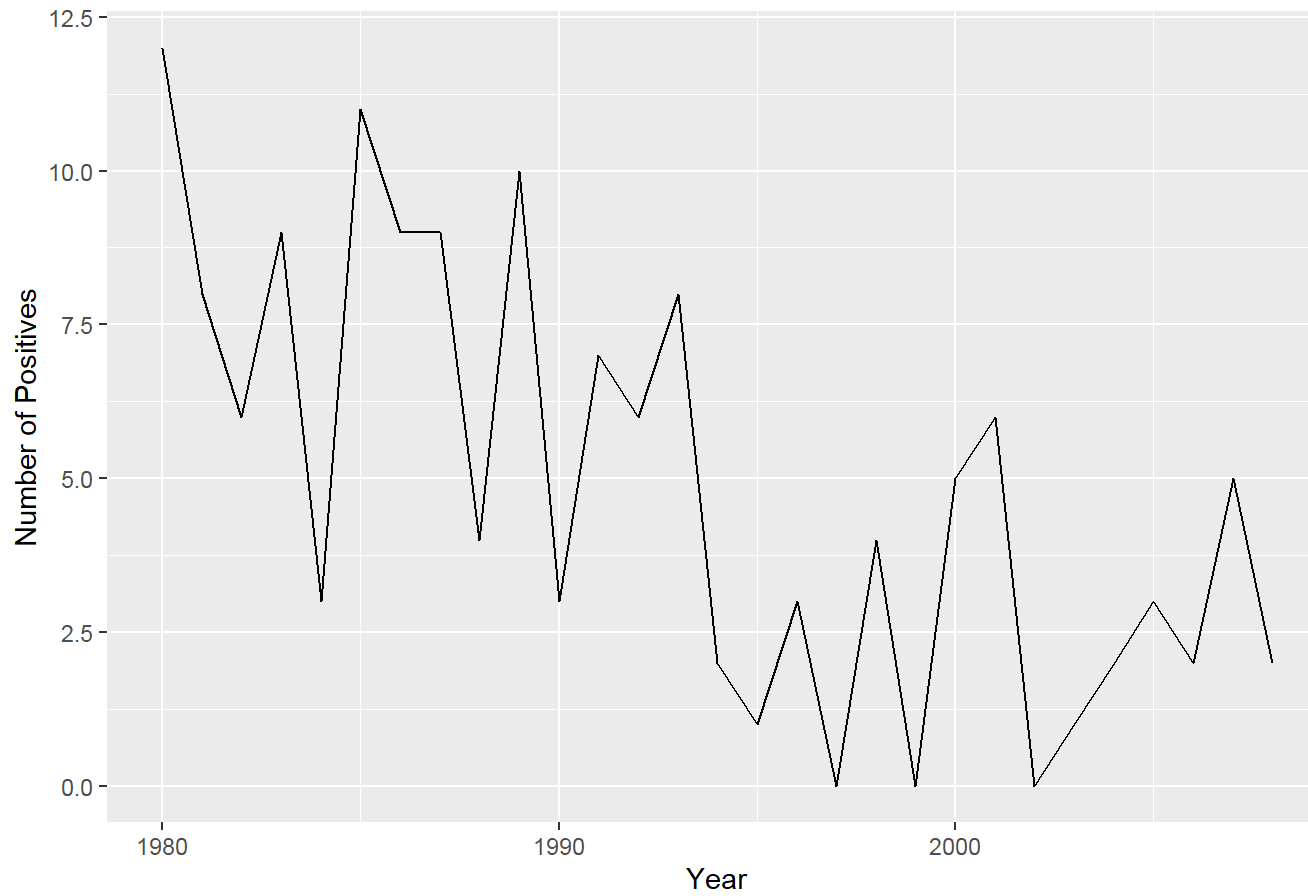
Step 4: Explore how the data changes over time

J. These data were collected in a period of several decades (1980-2013). You can thus observe changes over time with the help of a line chart.

Create a **line chart**, with **year** on the X-axis and **new_sp_m014** on the Y-axis.

```
line.plot<- ggplot(tb.can, aes(x=year, y=new_sp_m014))+  
  geom_line() + labs(title="Line Plot of Positive Pulmonary Smears: Males Ages 0 to 14", x="Year", y="Number of Positives" )  
line.plot
```

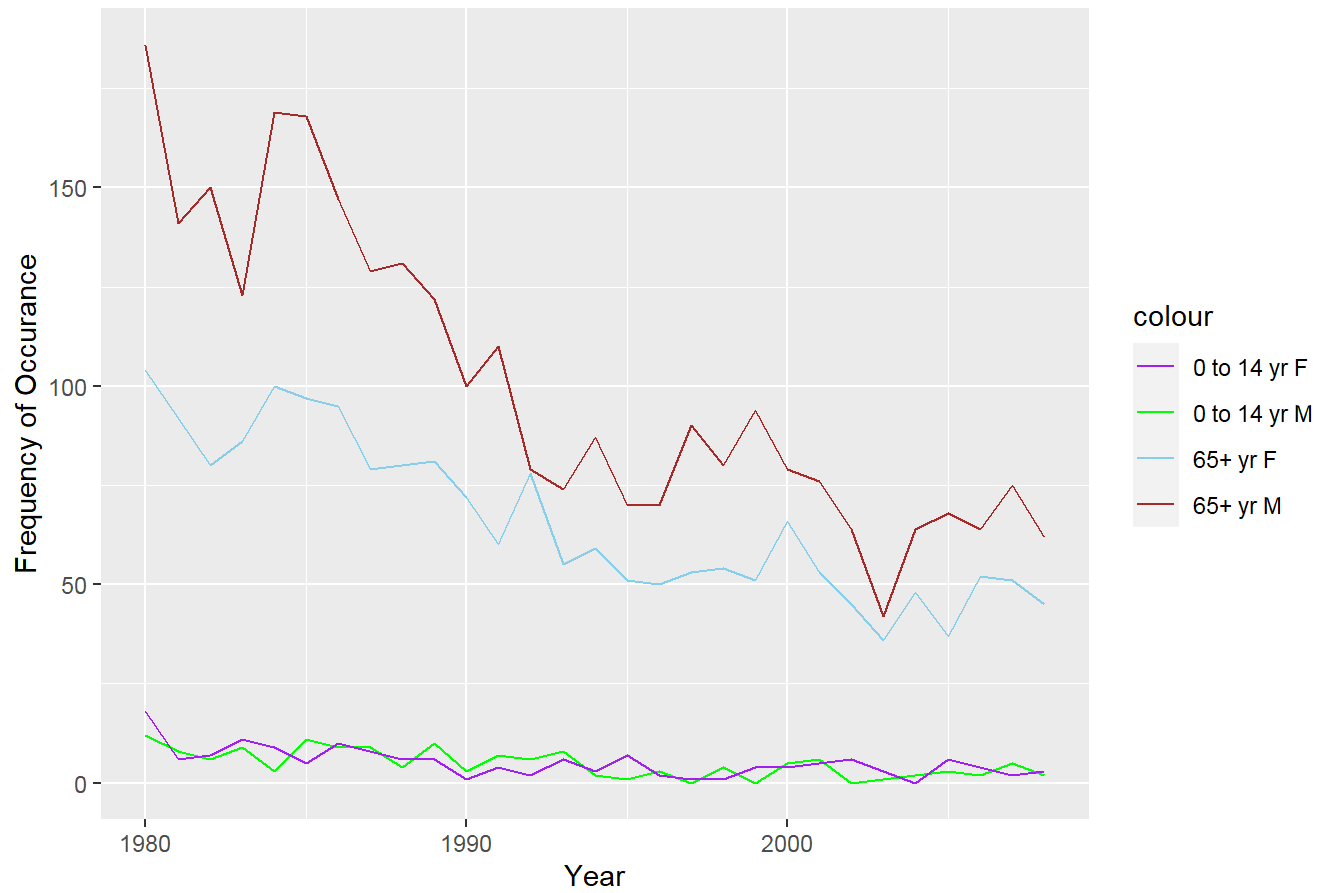
Line Plot of Positive Pulmonary Smears: Males Ages 0 to 14



K. Next, create similar graphs for each of the other three variables. Change the **color** of the line plots (any color you want).

```
line.plot2<- ggplot(tb.can) +
  geom_line(aes(x=year, y=new_sp_m014, color="0 to 14 yr M"))+
  geom_line(aes(x=year, y=new_sp_f014, color="0 to 14 yr F"))+
  geom_line(aes(x=year, y=new_sp_m65, color="65+ yr M")) +
  geom_line(aes(x=year, y=new_sp_f65, color="65+ yr F"))+
  labs(title="Line Plot of Positive Pulmonary Smears Age Comparison", x="Year", y="Frequency of Occurance")+
  scale_color_manual(values=c("0 to 14 yr M"="green", "0 to 14 yr F"="purple", "65+ yr M"="brown", "65+ yr F"="skyblue"))
line.plot2
```

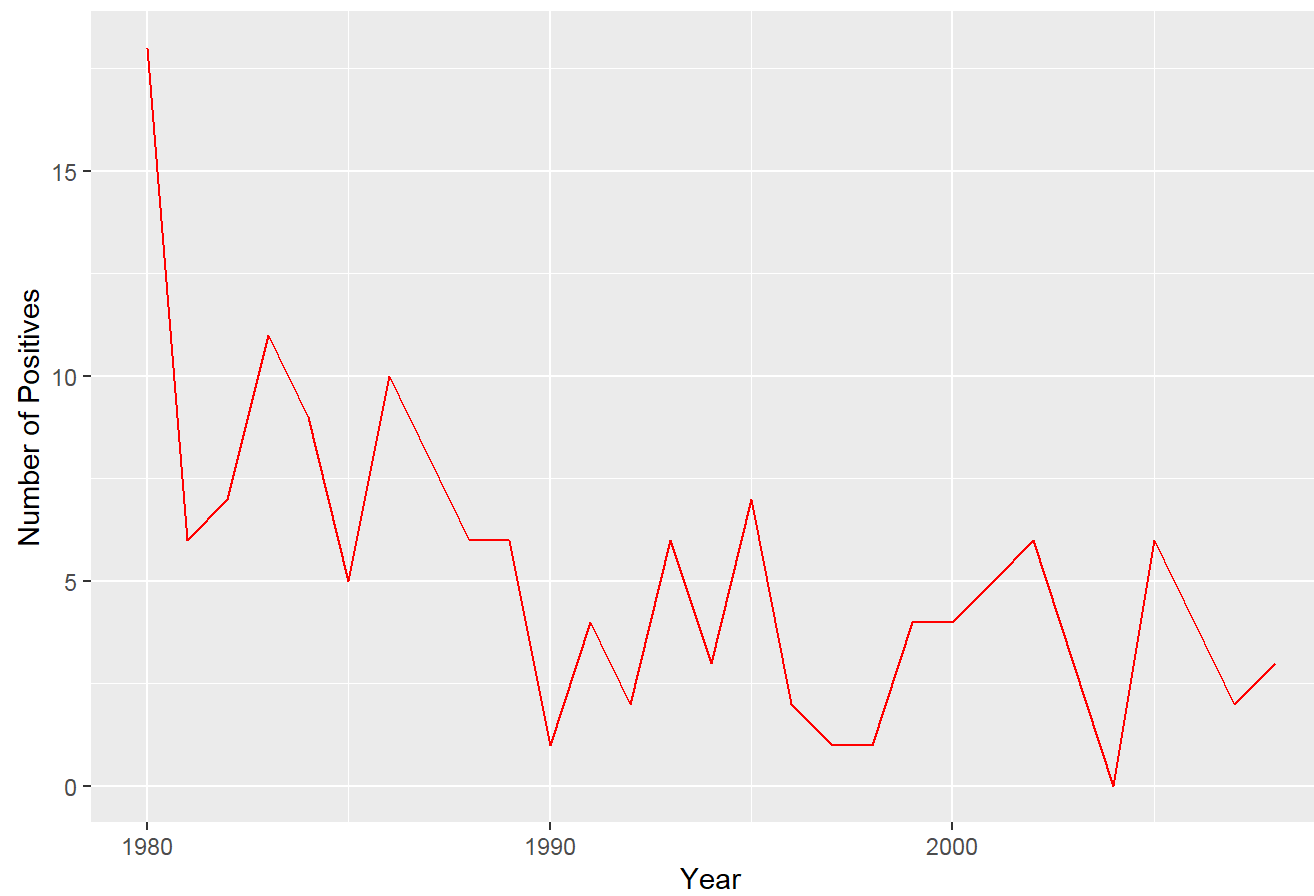
Line Plot of Positive Pulmonary Smears Age Comparison



Crap, that says graph(s) not graph. Well, copy and paste to the rescue

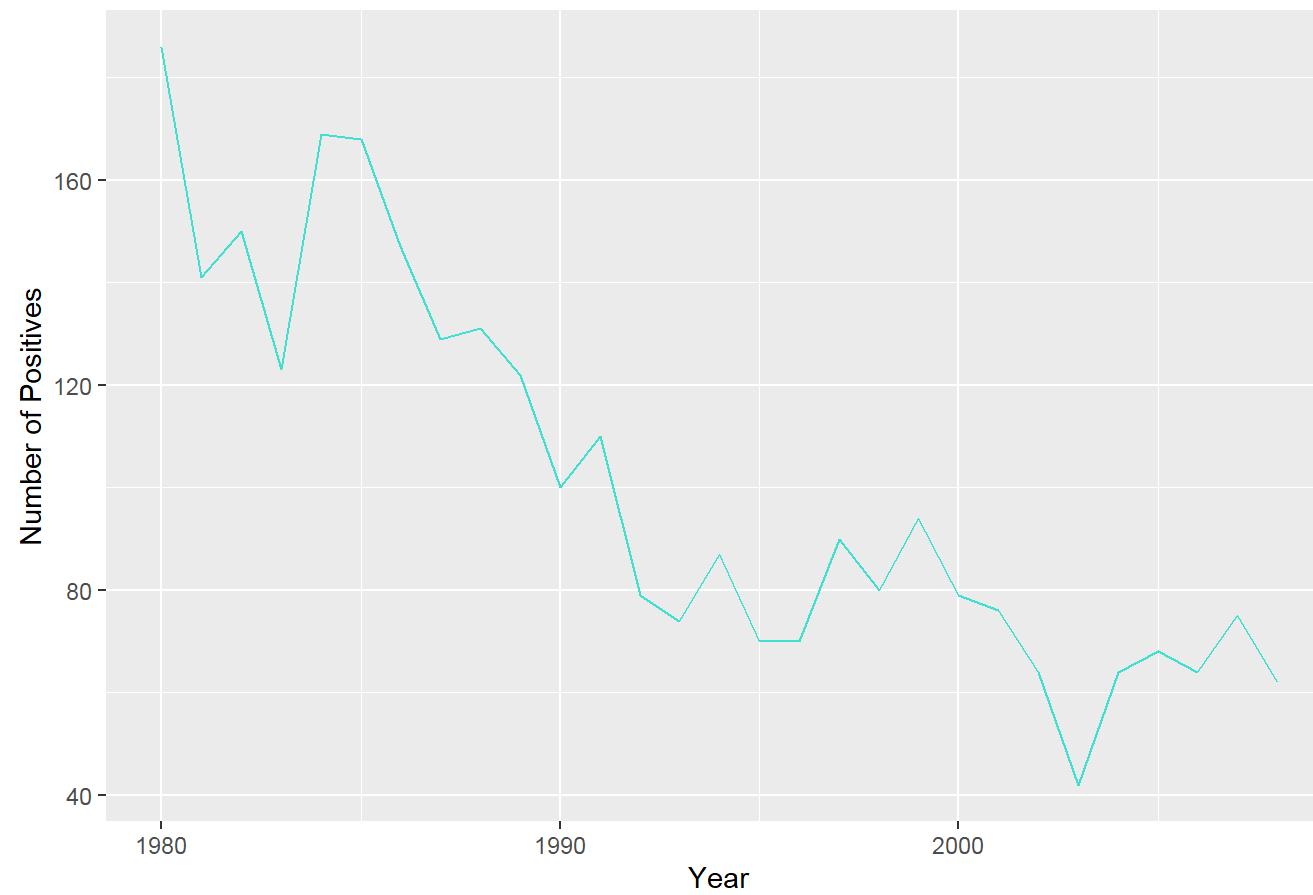
```
line.plot3<- ggplot(tb.can, aes(x=year, y=new_sp_f014))+
  geom_line(color="red") + labs(title="Line Plot of Positive Pulmonary Smears: Females Ages 0 to 14", x="Year", y="Number of
Positives" )
line.plot3
```

Line Plot of Positive Pulmonary Smears: Females Ages 0 to 14



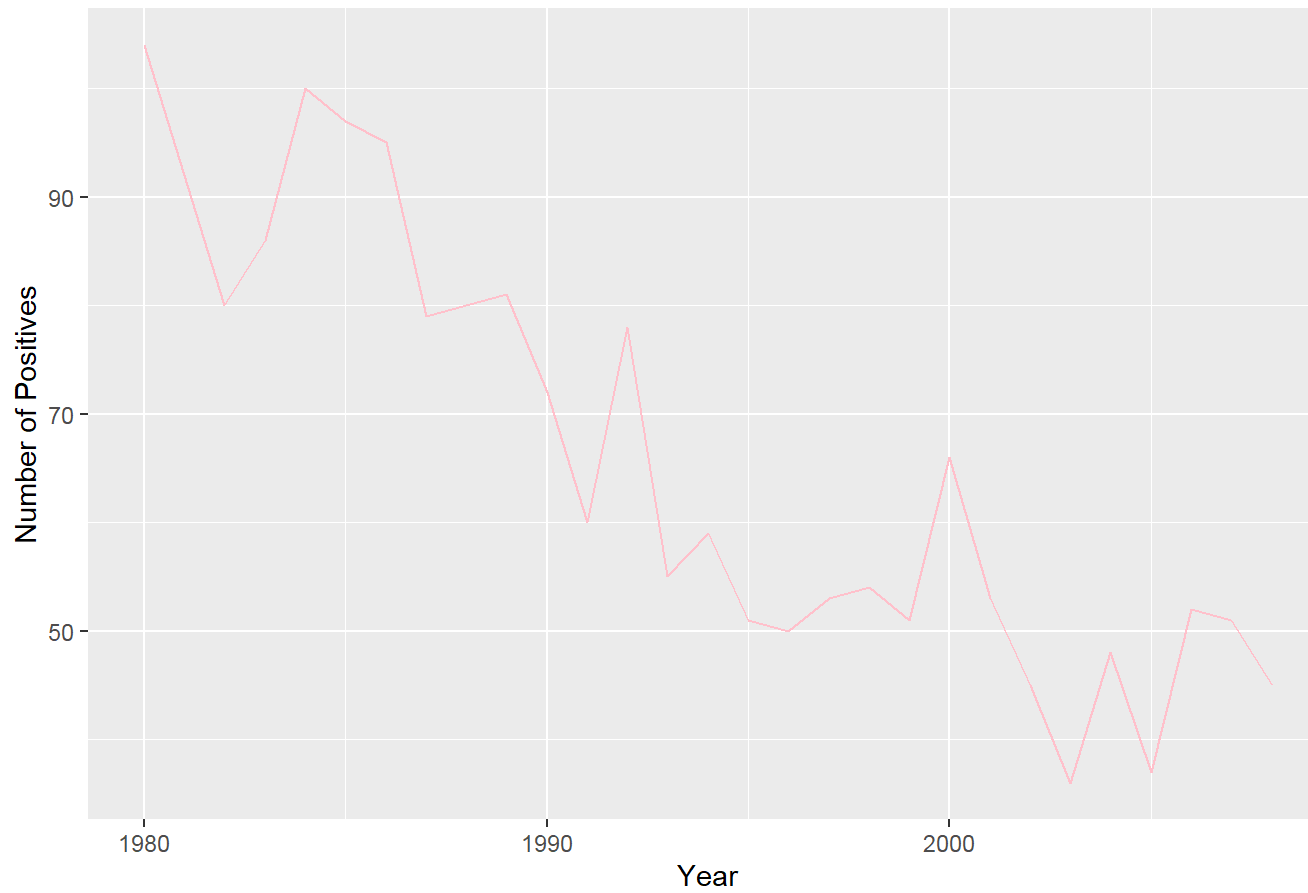
```
line.plot4<- ggplot(tb.can, aes(x=year, y=new_sp_m65))+  
  geom_line(color="turquoise") + labs(title="Line Plot of Positive Pulmonary Smears: Males Ages 65+", x="Year", y="Number of  
Positives" )  
line.plot4
```

Line Plot of Positive Pulmonary Smears: Males Ages 65+



```
line.plot5<- ggplot(tb.can, aes(x=year, y=new_sp_f65))+  
  geom_line(color="pink") + labs(title="Line Plot of Positive Pulmonary Smears: Females Ages 65+", x="Year", y="Number of Pos  
itives" )  
line.plot5
```

Line Plot of Positive Pulmonary Smears: Females Ages 65+



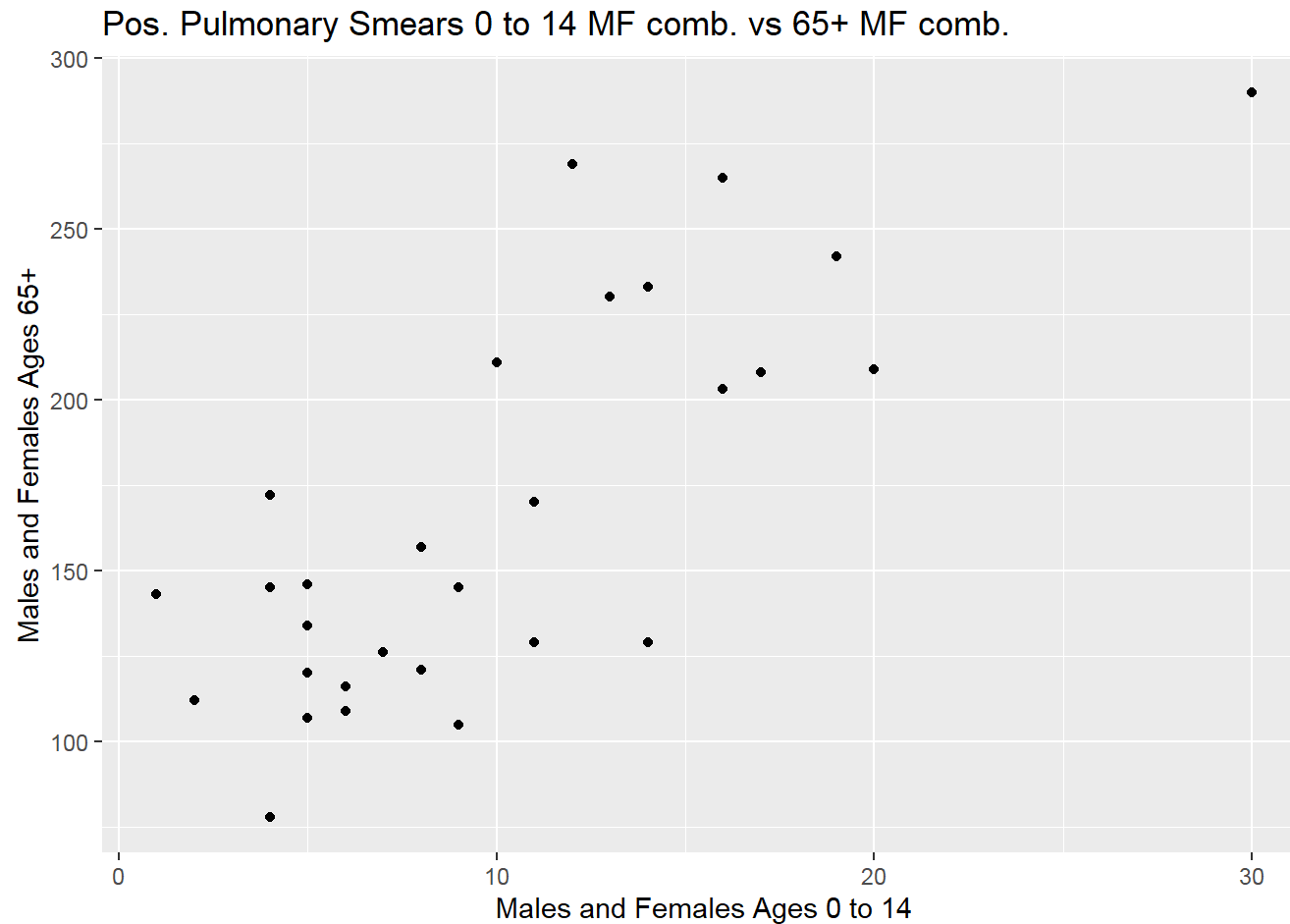
Individual plots

- L. Using vector math, create a new variable by combining the numbers from **new_sp_m014** and **new_sp_f014**. Save the resulting vector as a new variable in the **tbCan** df called **new_sp_combined014**. This new variable represents the number of positive pulmonary smear tests for male AND female children between the ages of 0 and 14 years of age. Do the same for SP tests among citizens 65 years of age and older and save the resulting vector in the **tbCan** variable called **new_sp_combined65**.

```
tb.can<- tb.can %>% mutate(new_sp_combined014 = new_sp_m014 + new_sp_f014)
tb.can<- tb.can %>% mutate(new_sp_combined65 = new_sp_m65 + new_sp_f65)
```

- M. Finally, create a **scatter plot**, showing **new_sp_combined014** on the x axis, **new_sp_combined65** on the y axis, and having the **color and size** of the point represent **year**.

```
scatter<-ggplot(tb.can, aes(x=new_sp_combined014, y=new_sp_combined65))+  
  geom_point()+  
  labs(title="Pos. Pulmonary Smears 0 to 14 MF comb. vs 65+ MF comb.", x="Males and Females Ages 0 to 14", y="Males and Females Ages 65+")  
scatter
```



N. Interpret this visualization – what insight does it provide?

This visualization gives us an idea of the relationship between two variables. With it we can see the direction of the relationship. In relation to the data, the data suggest that tb is more prevalent in older populations, but with this alone we cannot speculate as to why.